In [1]:
```
# ============================================================================
# PHASE 12: Market-Aware Skill-Gap Engine
# ============================================================================
# Cell 1: Setup and Comprehensive Data Loading
# ============================================================================
import pandas as pd
import numpy as np
import json
from pathlib import Path
from collections import Counter

# --- Define a simple class for consistent path management ---
class SimplePathResolver:
    def __init__(self, base_output_dir="outputs"):
        self.base = Path(base_output_dir)
    def phase_dir(self, phase_num):
        return self.base / f"phase{phase_num:02d}"

P = SimplePathResolver()
OUTDIR = P.phase_dir(12)
OUTDIR.mkdir(parents=True, exist_ok=True)

print("\n" + "="*60)
print("Phase 12: Market-Aware Skill-Gap Engine")
print("="*60)
print(f"All outputs will be saved to: {OUTDIR}")

# --- STEP 1: LOAD ALL NECESSARY PRIMARY ARTIFACTS ---
print("\nLoading all necessary artifacts from previous phases...")
P1_DIR = P.phase_dir(1) / "tables"
P3_DIR = P.phase_dir(3) / "tables"
P4_DIR = P.phase_dir(4)

# 1. The clean, filtered IT jobs corpus from Phase 1 (our "market" data)
jobs_df = pd.read_csv(P1_DIR / "jobs_it.csv")
print(f"✓ Loaded {len(jobs_df)} IT jobs from Phase 1.")

# 2. The canonical skills for those jobs from Phase 3
job_skills_data = [json.loads(line) for line in open(P3_DIR / "jobs_skills.jsonl", 'r', encoding='utf-8')]
```

```python
job_skills_df = pd.DataFrame(job_skills_data)
# Ensure job_id is the same type for merging
jobs_df['job_id'] = jobs_df['job_id'].astype(str)
job_skills_df['job_id'] = job_skills_df['job_id'].astype(str)
# Merge skills into the main jobs DataFrame
jobs_df = pd.merge(jobs_df, job_skills_df, on='job_id', how='left')
print(f"✓ Merged canonical skills into the jobs DataFrame.")

# 3. The canonical skills for all résumés from Phase 3
resume_skills_data = [json.loads(line) for line in open(P3_DIR / "resumes_skills.jsonl", 'r', encoding='utf-8')]
resume_skills_df = pd.DataFrame(resume_skills_data)
print(f"✓ Loaded {len(resume_skills_df)} résumé skill lists from Phase 3.")

# 4. The full ontology data from Phase 4
ontology_df = pd.read_csv(P4_DIR / "role_skills_ontology.csv")
ontology_df['skills_ont'] = ontology_df['skills_ont'].apply(json.loads)
# Create a simple dictionary for quick lookups
role_id_to_title = pd.Series(ontology_df.role_title.values, index=ontology_df.role_id).to_dict()
print(f"✓ Loaded {len(ontology_df)} ontology roles from Phase 4.")

print("\n" + "="*60)
print("SETUP COMPLETE: All data is loaded and ready.")
print("="*60)
```

```
============================================================
Phase 12: Market-Aware Skill-Gap Engine
============================================================
All outputs will be saved to: outputs\phase12

Loading all necessary artifacts from previous phases...
✓ Loaded 600 IT jobs from Phase 1.
✓ Merged canonical skills into the jobs DataFrame.
✓ Loaded 1200 résumé skill lists from Phase 3.
✓ Loaded 493 ontology roles from Phase 4.

============================================================
SETUP COMPLETE: All data is loaded and ready.
============================================================
```

In [2]:

```python
# ============================================================================
# Cell 2: Pre-computation: Building the Market Skills Database
```

```python
# ============================================================================
print("\n" + "="*60)
print("Building the Market Skills Database from Job Postings")
print("="*60)

# --- Define Role Family Keywords ---
# This is a heuristic to group job titles into broader categories.
ROLE_FAMILY_KEYWORDS = {
    'Data': ['data', 'analyst', 'scientist', 'engineer', 'business intelligence', 'bi'],
    'Software': ['software', 'developer', 'programmer', 'web', 'frontend', 'backend'],
    'DevOps/Cloud': ['devops', 'cloud', 'sre', 'platform', 'infrastructure', 'aws', 'azure', 'gcp'],
    'Security': ['security', 'cybersecurity', 'threat', 'pentest'],
    'Management': ['manager', 'lead', 'director', 'project manager'],
}

def get_role_family(title_str):
    """Assigns a role family based on keywords in the job title."""
    title_lower = str(title_str).lower()
    for family, keywords in ROLE_FAMILY_KEYWORDS.items():
        if any(keyword in title_lower for keyword in keywords):
            return family
    return 'Other' # Default category

# --- Map jobs and ontology roles to these families ---
jobs_df['role_family'] = jobs_df['job_title'].apply(get_role_family)
ontology_df['role_family'] = ontology_df['role_title'].apply(get_role_family)

print("✓ Assigned role families to jobs and ontology roles.")
print("Job family distribution:\n", jobs_df['role_family'].value_counts())

# --- Calculate top skills per family from the job market data ---
market_skills_db = {}
N_TOP_SKILLS = 15 # The number of top market skills to consider for each family

for family in jobs_df['role_family'].unique():
    # Filter jobs belonging to this family
    family_jobs = jobs_df[jobs_df['role_family'] == family]

    # Create a flat list of all skills mentioned in this family's job posts
    all_skills = [skill for skill_list in family_jobs['skills'] for skill in skill_list]
```

```
    # Count the frequency of each skill
    skill_counts = Counter(all_skills)

    # Get the top N most common skills
    top_n_skills = [skill for skill, count in skill_counts.most_common(N_TOP_SKILLS)]

    market_skills_db[family] = set(top_n_skills)

print(f"\n✓ Created Market Skills Database. Found top {N_TOP_SKILLS} skills for {len(market_skills_db)} families.")
print("\nExample - Top skills for 'Data' family:", list(market_skills_db.get('Data', []))[:5], "...")
```

```
============================================================
Building the Market Skills Database from Job Postings
============================================================
✓ Assigned role families to jobs and ontology roles.
Job family distribution:
 role_family
Other           288
Management      208
Data             89
Software          8
Security          4
DevOps/Cloud      3
Name: count, dtype: int64

✓ Created Market Skills Database. Found top 15 skills for 6 families.

Example - Top skills for 'Data' family: ['documentation', 'reporting', 'analysis', 'teams', 'software'] ...
```

In [9]:

```
# ============================================================================
# Cell 3 (ROBUST & CORRECTED): The Enhanced Market-Aware Skill-Gap Engine
# ============================================================================

def analyze_market_aware_skill_gap(candidate_skills_set, target_role_id_int, ontology_df, market_skills_db):
    """
    Analyzes the skill gap for a given set of candidate skills against a target role.
    This version takes the skills directly, avoiding any brittle ID lookups.
    """
    print("\n" + "="*70)
    print(f"      MARKET-AWARE SKILL GAP ANALYSIS")
    print("="*70)
```

```python
    # 1. Get information about the target role from the ontology
    try:
        target_role_info = ontology_df[ontology_df['role_id'] == target_role_id_int].iloc[0]
        ontology_core_skills = set(target_role_info['skills_ont'])
        target_role_title = target_role_info['role_title']
        target_role_family = target_role_info['role_family']
        print(f"Target Role: '{target_role_title}' (Family: {target_role_family})")
    except IndexError:
        print(f"[ERROR] Could not find target_role_id: {target_role_id_int} in the ontology.")
        return

    # 2. Get the in-demand skills from our pre-computed market database
    market_demand_skills = market_skills_db.get(target_role_family, set())

    # 3. Calculate the different categories of missing skills
    missing_core_skills = ontology_core_skills - candidate_skills_set
    missing_market_skills = market_demand_skills - candidate_skills_set

    # The full set of required skills is the union of both
    all_required_skills = ontology_core_skills.union(market_demand_skills)
    total_missing = all_required_skills - candidate_skills_set

    print(f"\nCandidate has {len(candidate_skills_set)} unique skills.")

    if not total_missing:
        print("\n✓ EXCEPTIONAL! This candidate possesses all core AND top market skills for this role.")
    else:
        print(f"\n! Skill Gap Identified: {len(total_missing)} skills are missing in total.")

        if missing_core_skills:
            print("\n--- Missing Core Foundational Skills (from Ontology) ---")
            for i, skill in enumerate(sorted(list(missing_core_skills))[:10], 1):
                print(f"    {i}. {skill}")

        # We only show market skills that are not already listed as missing core skills
        unique_missing_market = missing_market_skills - missing_core_skills
        if unique_missing_market:
            print("\n--- Missing In-Demand Market Skills (from Job Postings) ---")
            for i, skill in enumerate(sorted(list(unique_missing_market))[:10], 1):
                print(f"    {i}. {skill}")
```

```
    return

print("✓ Enhanced skill-gap engine is defined and ready.")
```

```
✓ Enhanced skill-gap engine is defined and ready.
```

In [11]:

```
# ==============================================================================
# Cell 4 (ROBUST & CORRECTED): Re-running the Case Studies
# ==============================================================================

# --- Start of the FINAL, ROBUST FIX ---

# STEP 1: LOAD THE SKILLS DATA THAT WAS MISSING
# This step is crucial and must happen before we can assemble the data for the case study.
print("Loading resume skills data from Phase 3...")
P3_DIR = P.phase_dir(3) / "tables"
try:
    skills_data = [json.loads(line) for line in open(P3_DIR / "resumes_skills.jsonl", 'r', encoding='utf-8')]
    skills_df = pd.DataFrame(skills_data)
    print(f"✓ Successfully loaded {len(skills_df)} skill lists.")
except FileNotFoundError:
    print(f"[ERROR] Could not find 'resumes_skills.jsonl'. Please ensure Phase 3 was run successfully.")
    raise

# STEP 2: RECREATE THE FINAL MODELING DATASET
# This ensures perfect consistency and uses our "single source of truth".
print("\nRecreating the final modeling dataset to get consistent data...")
P7_DIR = P.phase_dir(7)
labels_df = pd.read_csv(P7_DIR / "resume_labels.csv")

# Now that skills_df is loaded, this master_df creation will succeed.
master_df = pd.DataFrame({
    'resume_id': labels_df['resume_id'].astype(str),
    'label_confidence': labels_df['label_confidence'],
    'ontology_role_id': labels_df['ontology_role_id'],
    'skills_text': skills_df['skills'].apply(lambda x: ' '.join(x))
})

# Filter and clean just like in Phase 8
model_df = master_df[master_df['label_confidence'] == 'high_confidence'].copy()
class_counts = model_df['ontology_role_id'].value_counts()
```

```python
single_member_classes = class_counts[class_counts < 2].index
model_df = model_df[~model_df['ontology_role_id'].isin(single_member_classes)]
print(f"✓ Final modeling set of {len(model_df)} records is ready.")

# --- End of the FINAL, ROBUST FIX ---


# --- DEMONSTRATION on Case Study #2 (The "Model Disagreement") ---
# This remains the more interesting case to analyze.
# In Phase 11, the "True Label" for this case was 'PROJECT MANAGER' (ID 448)

# Find the specific record in our clean model_df programmatically.
try:
    case_study_record = model_df[model_df['ontology_role_id'] == 448].iloc[0]

    # 1. Get the candidate's skills DIRECTLY from the DataFrame
    candidate_skills = set(case_study_record['skills_text'].split())

    # 2. Get the target role ID
    target_role_id = int(case_study_record['ontology_role_id'])

    # 3. Run the new, more robust analysis function
    analyze_market_aware_skill_gap(
        candidate_skills_set=candidate_skills,
        target_role_id_int=target_role_id,
        ontology_df=ontology_df,
        market_skills_db=market_skills_db
    )
except IndexError:
    print("\n[INFO] Could not find the example case study record (ID 448) in the final modeling set.")
    print("This can happen due to random splits if the record is not in the training/validation data. The logic is correct.")
```

```
Loading resume skills data from Phase 3...
✓ Successfully loaded 1200 skill lists.

Recreating the final modeling dataset to get consistent data...
✓ Final modeling set of 216 records is ready.

======================================================================
      MARKET-AWARE SKILL GAP ANALYSIS
======================================================================
Target Role: 'PROJECT MANAGER' (Family: Management)

Candidate has 7 unique skills.

! Skill Gap Identified: 19 skills are missing in total.

--- Missing Core Foundational Skills (from Ontology) ---
    1. Communication
    2. Execution
    3. Planning
    4. Project Management

--- Missing In-Demand Market Skills (from Job Postings) ---
    1. analysis
    2. communication
    3. compliance
    4. etc.
    5. implementation
    6. leadership
    7. planning
    8. project management
    9. reporting
    10. sales
```